Neural Quantum States for Lattice Spin Systems

A pedagogical tour of the `NQS` package

`NQS` project documentation

April 23, 2026

Abstract

This note explains, from first principles, every line of physics and machine learning behind the small JAX/Flax library hosted at `github.com/ze-bang/NQS`. We start from the variational principle for the quantum many-body ground state, derive the stochastic estimator for the energy and its gradient, develop the kernel-trick natural gradient (**MinSR**), and finally build the three architectures shipped in the package: a translation-invariant convolutional network (**CNN**), a fully space-group-equivariant **GCNN**, and a Vision-Transformer ansatz (**ViT**). Each section ends with a pointer to the file in the repository that implements it.

Contents

1 The problem and the variational principle **2**

2 The local-energy estimator **2**

3 Sampling: Metropolis on the spin lattice **3**

4 Optimisation 1 — the energy gradient **4**

5 Optimisation 2 — Stochastic Reconfiguration and MinSR **4**
- 5.1 The Fisher–Rao geometry of $|\psi_{\boldsymbol{\theta}}\rangle$ . . . 4
- 5.2 Why P^3 is too slow . . . 4
- 5.3 The MinSR kernel trick (Chen & Heyl, 2024) . . . 5
- 5.4 Computing the Jacobian O . . . 5

6 The ansätze: from CNN to GCNN **5**
- 6.1 Real-/complex-valued outputs . . . 5
- 6.2 The plain CNN (translation only) . . . 6
- 6.3 The Vision Transformer (ViT) . . . 6
- 6.4 Group-equivariant CNN (GCNN) . . . 6

7 Putting it together: the training loop **7**

8 Validation: comparison with exact diagonalisation **8**

9 Code map **8**

References **8**

1 The problem and the variational principle

We are interested in the ground state of a spin-1/2 Hamiltonian on a finite lattice with N sites,

$$\hat{H} \,:\, \mathcal{H} \to \mathcal{H}, \qquad \mathcal{H} = \bigotimes_{i=1}^{N} \mathbb{C}^2, \tag{1}$$

of dimension $D = 2^N$. We use the computational basis labelled by configurations $\boldsymbol{s} = (s_1, \ldots, s_N) \in \{-1, +1\}^N$, eigenstates of σ_i^z.

For $N \gtrsim 30$ the basis is too large to store, let alone diagonalise. The **variational principle** circumvents this by parametrising the wavefunction with a tractable function $\psi_{\boldsymbol{\theta}}$ of the configuration:

$$|\psi_{\boldsymbol{\theta}}\rangle \;=\; \sum_{\boldsymbol{s}} \psi_{\boldsymbol{\theta}}(\boldsymbol{s})\, |\boldsymbol{s}\rangle, \qquad \psi_{\boldsymbol{\theta}} : \{-1, +1\}^N \to \mathbb{C}. \tag{2}$$

By the Rayleigh–Ritz inequality, for any nonzero $\psi_{\boldsymbol{\theta}}$,

$$\boxed{E(\boldsymbol{\theta}) \;\equiv\; \frac{\langle \psi_{\boldsymbol{\theta}} | \hat{H} | \psi_{\boldsymbol{\theta}} \rangle}{\langle \psi_{\boldsymbol{\theta}} | \psi_{\boldsymbol{\theta}} \rangle} \;\geq\; E_0.} \tag{3}$$

Minimising $E(\boldsymbol{\theta})$ over $\boldsymbol{\theta} \in \mathbb{R}^P$ is therefore a principled strategy. The two questions that occupy the rest of this note are: *which* $\psi_{\boldsymbol{\theta}}$ to choose, and *how* to minimise (3) when summing 2^N configurations is impossible.

Scope. The package implements two paradigmatic models. The *transverse-field Ising model* (TFIM)

$$\hat{H}_{\text{TFIM}} = -J \sum_{\langle ij \rangle} \sigma_i^z \sigma_j^z - h \sum_i \sigma_i^x, \tag{4}$$

and the (frustrated) Heisenberg J_1–J_2 model

$$\hat{H}_{\text{Heis}} = J_1 \sum_{\langle ij \rangle} \vec{S}_i \cdot \vec{S}_j + J_2 \sum_{\langle\langle ij \rangle\rangle} \vec{S}_i \cdot \vec{S}_j, \tag{5}$$

both with periodic boundary conditions. Both have a sparse local representation in the computational basis, which is what we exploit next.

Code: `nqs/lattices.py`, `nqs/hamiltonians.py`.

2 The local-energy estimator

A direct evaluation of (3) would require the full sum

$$E(\boldsymbol{\theta}) = \frac{\sum_{\boldsymbol{s},\boldsymbol{s}'} \overline{\psi_{\boldsymbol{\theta}}(\boldsymbol{s})}\, \langle \boldsymbol{s} | \hat{H} | \boldsymbol{s}' \rangle\, \psi_{\boldsymbol{\theta}}(\boldsymbol{s}')}{\sum_{\boldsymbol{s}} |\psi_{\boldsymbol{\theta}}(\boldsymbol{s})|^2}. \tag{6}$$

The trick is to factor the numerator so that the same probability $|\psi_{\boldsymbol{\theta}}(\boldsymbol{s})|^2$ appears in both:

$$E(\boldsymbol{\theta}) = \sum_{\boldsymbol{s}} \frac{|\psi_{\boldsymbol{\theta}}(\boldsymbol{s})|^2}{Z(\boldsymbol{\theta})} \underbrace{\sum_{\boldsymbol{s}'} \langle \boldsymbol{s} | \hat{H} | \boldsymbol{s}' \rangle \frac{\psi_{\boldsymbol{\theta}}(\boldsymbol{s}')}{\psi_{\boldsymbol{\theta}}(\boldsymbol{s})}}_{E_{\text{loc}}(\boldsymbol{s})} \;=\; \mathbb{E}_{\boldsymbol{s} \sim p_{\boldsymbol{\theta}}}\big[E_{\text{loc}}(\boldsymbol{s})\big], \tag{7}$$

$$p_{\boldsymbol{\theta}}(\boldsymbol{s}) \;=\; \frac{|\psi_{\boldsymbol{\theta}}(\boldsymbol{s})|^2}{Z(\boldsymbol{\theta})}, \qquad Z(\boldsymbol{\theta}) = \sum_{\boldsymbol{s}} |\psi_{\boldsymbol{\theta}}(\boldsymbol{s})|^2. \tag{8}$$

The crucial observation is that for local Hamiltonians the matrix element $\langle \boldsymbol{s}|\hat{H}|\boldsymbol{s}'\rangle$ is non-zero only for an $\mathcal{O}(N)$-sized set of $\boldsymbol{s}'$ for each $\boldsymbol{s}$. Therefore the **local energy** $E_{\mathrm{loc}}(\boldsymbol{s})$ is computable in $\mathcal{O}(N)$ forward passes through the network, and an unbiased estimator of E comes from a Monte Carlo average over N_s samples:

$$\widehat{E} = \frac{1}{N_s}\sum_{k=1}^{N_s} E_{\mathrm{loc}}(\boldsymbol{s}^{(k)}), \qquad \boldsymbol{s}^{(k)} \overset{\text{i.i.d.}}{\sim} p_{\boldsymbol{\theta}}. \tag{9}$$

TFIM connections. The diagonal piece of (4) is $-J\sum_{\langle ij\rangle} s_i s_j$. Each σ_i^x flips spin i, contributing one off-diagonal term per site,

$$\langle \boldsymbol{s}^{(i)}|\hat{H}_{\mathrm{TFIM}}|\boldsymbol{s}\rangle = -h, \qquad \boldsymbol{s}^{(i)} = (\dots, -s_i, \dots). \tag{10}$$

So $E_{\mathrm{loc}}(\boldsymbol{s})$ is a sum of one diagonal term and N ratios $\psi_{\boldsymbol{\theta}}(\boldsymbol{s}^{(i)})/\psi_{\boldsymbol{\theta}}(\boldsymbol{s})$.

Heisenberg connections. Writing $\vec{S} = \frac{1}{2}\vec{\sigma}$,

$$\vec{S}_i \cdot \vec{S}_j = \tfrac{1}{4}\sigma_i^z\sigma_j^z + \tfrac{1}{2}(\sigma_i^+\sigma_j^- + \sigma_i^-\sigma_j^+), \tag{11}$$

the off-diagonal piece swaps two opposite spins; only *antiparallel* bonds give an off-diagonal contribution. So $E_{\mathrm{loc}}(\boldsymbol{s})$ has at most |bonds| ratios.

Marshall sign. On bipartite lattices the unfrustrated Heisenberg model has a strictly positive ground state in a rotated basis, $\psi(\boldsymbol{s}) \mapsto (-1)^{N_{\uparrow,B}(\boldsymbol{s})}\psi(\boldsymbol{s})$. The package implements this rotation by flipping the sign of every off-diagonal matrix element, which dramatically simplifies the function the network has to learn (no sign structure to discover when $J_2 = 0$).

Code: `Hamiltonian.local_terms()` and `nqs.vmc.local_energy()`.

3 Sampling: Metropolis on the spin lattice

We need samples from $p_{\boldsymbol{\theta}}(\boldsymbol{s}) \propto |\psi_{\boldsymbol{\theta}}(\boldsymbol{s})|^2$. Direct sampling is intractable, but Metropolis–Hastings only needs *ratios*, and the network gives us

$$\frac{p_{\boldsymbol{\theta}}(\boldsymbol{s}')}{p_{\boldsymbol{\theta}}(\boldsymbol{s})} = \exp[2\,\mathrm{Re}(\log\psi_{\boldsymbol{\theta}}(\boldsymbol{s}') - \log\psi_{\boldsymbol{\theta}}(\boldsymbol{s}))]. \tag{12}$$

Local-flip kernel. For TFIM (which has no conservation law), propose flipping a single random spin and accept with probability $\min(1, p_{\boldsymbol{\theta}}(\boldsymbol{s}')/p_{\boldsymbol{\theta}}(\boldsymbol{s}))$.

Exchange kernel. For Heisenberg in the $S^z_{\mathrm{tot}}=0$ sector, propose swapping a random nearest-neighbour pair of opposite spins. This kernel automatically conserves the total magnetisation, so we never leave the relevant Hilbert-space sector.

Both kernels run `n_chains` chains in parallel inside a single `jax.lax.scan`, with `n_sweeps` = 2 fully re-randomising sweeps between recorded samples.

Code: `nqs/sampler.py`.

4 Optimisation 1 — the energy gradient

The naive route is to differentiate $\widehat{E}$ in (9). With $\log \psi_{\boldsymbol{\theta}}(\boldsymbol{s})$ the natural object the network outputs, the well-known result is[1]

$$\partial_{\theta_i} E = 2 \text{ Re } \mathbb{E}_{p_{\boldsymbol{\theta}}} \left[(E_{\text{loc}}(\boldsymbol{s}) - \mathbb{E}[E_{\text{loc}}]) \, \overline{\partial_{\theta_i} \log \psi_{\boldsymbol{\theta}}(\boldsymbol{s})} \right]. \tag{13}$$

The mean subtraction is a free **control variate**: it does not change the expectation but reduces the Monte Carlo variance, often by orders of magnitude. Plugging (13) into Adam already trains an NQS, but slowly — the next section explains why, and what to do about it.

Code: `nqs.optimizer.adam_step()`.

5 Optimisation 2 — Stochastic Reconfiguration and MinSR

5.1 The Fisher–Rao geometry of $|\psi_{\boldsymbol{\theta}}\rangle$

A vanilla gradient step $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla E$ is geometrically blind: it treats the parameter space as if it were Euclidean. But two parameter vectors that differ by a small Euclidean amount can correspond to very different quantum states (or to the same state, in directions tangent to re-parameterisations).

The right metric is the one inherited from the Hilbert space, the quantum Fisher-information / Fubini–Study metric. Define the per-sample log-derivatives

$$O_{ki} \;=\; \partial_{\theta_i} \log \psi_{\boldsymbol{\theta}}(\boldsymbol{s}^{(k)}), \qquad k = 1, \ldots, N_s, \; i = 1, \ldots, P, \tag{14}$$

and the (centred) covariance

$$S_{ij} = \mathbb{E}_{p_{\boldsymbol{\theta}}} \left[\overline{(\partial_i \log \psi_{\boldsymbol{\theta}})} \, (\partial_j \log \psi_{\boldsymbol{\theta}}) \right] - \overline{\mathbb{E}[\partial_i \log \psi_{\boldsymbol{\theta}}]} \, \mathbb{E}[\partial_j \log \psi_{\boldsymbol{\theta}}]. \tag{15}$$

Stochastic Reconfiguration (SR; Sorella, 1998) replaces the gradient step by the natural-gradient step

$$\boxed{\boldsymbol{\theta} \;\leftarrow\; \boldsymbol{\theta} - \eta \, (S + \lambda I)^{-1} \, g,} \tag{16}$$

where $g_i = \partial_{\theta_i} E$ is the energy gradient and λ is a small diagonal shift (Tikhonov regulariser). Geometrically, (16) chooses the parameter increment that, to leading order, minimises $E(\boldsymbol{\theta} + \delta\boldsymbol{\theta})$ subject to a fixed Fubini–Study distance $\|\delta\boldsymbol{\theta}\|_S^2$.

5.2 Why P^3 is too slow

Build the Monte Carlo estimator directly. With centred jacobian $\widetilde{O} = O - \bar{O}$ ($\bar{O}$ being the column mean) and centred local-energy residuals $\varepsilon_k = E_{\text{loc}}(\boldsymbol{s}^{(k)}) - \mathbb{E}[E_{\text{loc}}]$,

$$\widehat{S} = \frac{1}{N_s} \widetilde{O}^\dagger \widetilde{O} \in \mathbb{C}^{P \times P}, \qquad \widehat{g} = \frac{2}{N_s} \text{ Re}(\widetilde{O}^\dagger \boldsymbol{\varepsilon}). \tag{17}$$

The bottleneck is the linear system $(\widehat{S} + \lambda I) \, \delta\boldsymbol{\theta} = \widehat{g}$, of cost $\mathcal{O}(P^3)$. For modern networks P is 10^5–10^7, which is hopeless.

[1] Derived by differentiating $E = \mathbb{E}[E_{\text{loc}}]$ at fixed E_{loc} and noting that $\partial_i \log p = 2 \text{ Re} \, \partial_i \log \psi$. The mean subtraction term arises from the normalisation derivative.

5.3 The MinSR kernel trick (Chen & Heyl, 2024)

When $N_s \ll P$, the matrix $\widehat{S} = \frac{1}{N_s}\widetilde{O}^\dagger \widetilde{O}$ has rank at most N_s, so most of $\mathbb{C}^P$ is in its kernel. The minimum-norm solution of the (under-determined) system $\widetilde{O}\,\delta\boldsymbol{\theta} = \boldsymbol{\varepsilon}$ is exactly the SR step in the limit $\lambda \to 0$. Solving it by the kernel trick:

Proposition 1 (MinSR)**.** *For real parameters and complex* $\log \psi_{\boldsymbol{\theta}}$*, define the real "stacked" Jacobian and residual*

$$\mathbf{O} = \begin{pmatrix} \operatorname{Re}\widetilde{O} \\ \operatorname{Im}\widetilde{O} \end{pmatrix} \in \mathbb{R}^{2N_s \times P}, \qquad \boldsymbol{\epsilon} = \begin{pmatrix} \operatorname{Re}\boldsymbol{\varepsilon} \\ \operatorname{Im}\boldsymbol{\varepsilon} \end{pmatrix} \in \mathbb{R}^{2N_s}. \tag{18}$$

The (regularised) natural-gradient direction $\delta\boldsymbol{\theta} = (\widehat{S} + \lambda I)^{-1}\widehat{g}$ *equals*

$$\boxed{\delta\boldsymbol{\theta} \;=\; \mathbf{O}^\top (T + \lambda I_{2N_s})^{-1} \tfrac{1}{N_s}\boldsymbol{\epsilon}, \quad T \;=\; \tfrac{1}{N_s}\,\mathbf{O}\,\mathbf{O}^\top \;\in\; \mathbb{R}^{2N_s \times 2N_s}.} \tag{19}$$

Sketch. Use the Woodbury identity / push-through identity

$$(A^\top A + \lambda I_P)^{-1} A^\top = A^\top (AA^\top + \lambda I_{2N_s})^{-1} \tag{20}$$

with $A = \mathbf{O}/\sqrt{N_s}$. □ □

The cost is now $\mathcal{O}(N_s^2 P)$ to form T and $\mathcal{O}(N_s^3)$ to solve — linear in P. With $N_s \sim 10^3$ and $P \sim 10^6$ this is several orders of magnitude faster than the P^3 formulation, with mathematically identical results in the limit $\lambda \to 0$.

5.4 Computing the Jacobian O

For a network with real parameters $\boldsymbol{\theta}$ and complex output $\log \psi_{\boldsymbol{\theta}} = a + i\phi$, the column $O_{:,i}$ is the per-sample gradient $\partial_{\theta_i}(a + i\phi)$. We compute it with two `vmap(grad(...))` calls:

```
real = jax.vmap(jax.grad(lambda p, x: jnp.real(log_psi_single(p, x))),
                in_axes=(None, 0))(p_flat, s)
imag = jax.vmap(jax.grad(lambda p, x: jnp.imag(log_psi_single(p, x))),
                in_axes=(None, 0))(p_flat, s)
return real + 1j * imag            # shape (N_s, P)
```

Code: `nqs.vmc.minsr_update()`, `nqs.optimizer.minsr_step()`.

6 The ansätze: from CNN to GCNN

A neural quantum state is a function $\psi_{\boldsymbol{\theta}} : \{-1, +1\}^N \to \mathbb{C}$. The art is to choose a parametrisation that (i) is expressive enough to represent the target ground state, (ii) respects the symmetries of $\hat{H}$, and (iii) trains efficiently. The package ships three architectures.

6.1 Real-/complex-valued outputs

Each network outputs two real heads and combines them as $\log \psi_{\boldsymbol{\theta}}(\boldsymbol{s}) = a(\boldsymbol{s}) + i\,\phi(\boldsymbol{s})$. Working in log-space avoids numerical underflow for unnormalised configurations and, more importantly, makes the ratio $\psi_{\boldsymbol{\theta}}(\boldsymbol{s}')/\psi_{\boldsymbol{\theta}}(\boldsymbol{s}) = e^{\log \psi_{\boldsymbol{\theta}}(\boldsymbol{s}') - \log \psi_{\boldsymbol{\theta}}(\boldsymbol{s})}$ that appears in E_{loc} trivially stable.

6.2 The plain CNN (translation only)

Periodic-padded convolutions are translation-equivariant: shifting the input by one site shifts every feature map by one site. After a final *sum pooling* over the spatial dimensions and a dense readout to the two scalar heads, the entire network is translation-*invariant*:

$$\log \psi_{\boldsymbol{\theta}}(T_{\boldsymbol{a}} \boldsymbol{s}) \;=\; \log \psi_{\boldsymbol{\theta}}(\boldsymbol{s}) \qquad \forall\, \boldsymbol{a} \in \mathbb{Z}_L \times \mathbb{Z}_L. \tag{21}$$

This already captures the most important symmetry of the lattice. But it ignores point-group symmetries (rotations by 90° and reflections), which the ground state of (5) also possesses.

6.3 The Vision Transformer (ViT)

Following Viteritti, Rende and Becca (PRL 2023), partition the $L \times L$ lattice into $(L/p) \times (L/p)$ patches of size $p \times p$, embed each patch with a learned linear map plus a positional embedding, and feed the resulting tokens through a stack of multi-head self-attention blocks. Sum pooling over tokens yields a permutation-invariant scalar; the dense readout then produces a and ϕ.

ViT is permutation-equivariant over patches but is not lattice-symmetric out of the box — the positional embeddings are learned, not enforced. In practice it nevertheless reaches very competitive accuracies on frustrated lattices because of its flexibility.

6.4 Group-equivariant CNN (GCNN)

The GCNN bakes the *full space group* $G = T \rtimes P$ of the lattice (with P the point group, here C_{4v} for the square lattice) into the architecture by construction. Following Roth & MacDonald (PRB 2021):

1. The group as permutations. For each $g \in G$, let $\pi_g \in S_N$ be its action on the N sites, $g \cdot \boldsymbol{s} = (s_{\pi_g(1)}, \ldots, s_{\pi_g(N)})$. The package builds this representation explicitly in `nqs/groups.py` and verifies the group axioms:

$$\text{closure:} \qquad \mu(g,h) := g \cdot h \in G, \quad \forall g,h, \tag{22}$$

$$\text{identity:} \qquad \mu(e,g) = \mu(g,e) = g, \tag{23}$$

$$\text{inverse:} \qquad \mu(g,g^{-1}) = e, \tag{24}$$

$$\text{associativity:} \qquad \mu(\mu(g,h),k) = \mu(g,\mu(h,k)). \tag{25}$$

The Cayley table μ and the inverse table $g \mapsto g^{-1}$ are precomputed once and stored as integer arrays of shapes $|G| \times |G|$ and $|G|$.

2. Lifting layer. The first layer "lifts" the input to a feature map indexed by g:

$$f^{(0)}_{g,c}(\boldsymbol{s}) \;=\; \sigma\Big(W_c \cdot (g \cdot \boldsymbol{s})\Big), \qquad g \in G,\; c = 1, \ldots, C_0. \tag{26}$$

Concretely, for each g we permute the input via π_g and apply a shared dense map W_c. The resulting tensor has shape $(B,\, |G|,\, C_0)$.

3. Group convolutions. The hidden layers act on functions $f : G \to \mathbb{R}^C$. A **group convolution** with kernel $K : G \to \mathbb{R}^{C \times C'}$ is

$$\big(K * f\big)_g \;=\; \sum_{h \in G} K_{gh^{-1}}\, f_h, \qquad K_{gh^{-1}}\, f_h \in \mathbb{R}^{C'}. \tag{27}$$

This is the natural generalisation of a 2D convolution, replacing the shift group $\mathbb{Z}^2$ by a general group G.

Proposition 2 (Equivariance of group convolution)**.** *For every $g_0 \in G$, let L_{g_0} act on functions by $(L_{g_0} f)_g = f_{g_0^{-1} g}$. Then*

$$K * (L_{g_0} f) \;=\; L_{g_0}\,(K * f). \tag{28}$$

Proof. $(K * (L_{g_0} f))_g \;=\; \sum_h K_{gh^{-1}}\, f_{g_0^{-1} h} \;=\; \sum_{h'} K_{g(g_0 h')^{-1}}\, f_{h'} \;=\; \sum_{h'} K_{(g_0^{-1} g) h'^{-1}}\, f_{h'} \;=\; (K * f)_{g_0^{-1} g} = (L_{g_0}(K * f))_g$. □ □

So an arbitrary stack of group convolutions $f^{(\ell+1)} = \sigma(K^{(\ell)} * f^{(\ell)})$ preserves the action of G on the feature map.

4. Implementation as gather + einsum. Define the *shift index tensor*

$$T_{g,h} \;=\; g \cdot h^{-1}, \qquad T \in \{0, \dots, |G| - 1\}^{|G| \times |G|}, \tag{29}$$

so that (27) becomes

$$f^{(\ell+1)}_{b,g,c'} \;=\; \sigma\Big(\frac{1}{\sqrt{|G|}} \sum_{h,c} K^{(\ell)}_{T_{g,h},\,c,c'}\; f^{(\ell)}_{b,h,c}\Big). \tag{30}$$

This is one `gather` (along the kernel's group axis) and one `einsum "bhc,ghcd->bgd"`. The cost is $\mathcal{O}(B\,|G|^2\,CC')$ per layer; for $L = 4,\ G = T \times C_{4v}$ we have $|G| = 128$, which is cheap. The relevant snippet:

```
T  = mult[:, inv]                    # (G, G)  shift table
Kg = kernel[T]                       # (G, G, C_in, C_out)
x  = jnp.einsum("bhc,ghcd->bgd", x, Kg) / jnp.sqrt(G)
```

5. Character / irrep projection. After L group convolutions the per-sample tensor $f^{(L)} \in \mathbb{R}^{|G| \times C_L}$ transforms in the regular representation of G. To get a G-invariant scalar, project onto the trivial irrep:

$$\log \psi_{\boldsymbol{\theta}}(\boldsymbol{s}) \;=\; \frac{1}{|G|} \sum_{g \in G} \chi_0(g)\; \big[a_{\text{head}}(f^{(L)}_g) + i\,\phi_{\text{head}}(f^{(L)}_g)\big], \tag{31}$$

with $\chi_0(g) \equiv 1$ (the trivial character). To target a different ground-state symmetry sector, replace χ_0 by the character of the appropriate one-dimensional irrep — the rest of the architecture remains unchanged. By orthogonality of characters this recovers the appropriate projector $\Pi_{\text{irrep}}\,|\psi_{\boldsymbol{\theta}}\rangle$.

6. Empirical check. The implementation is tested directly in code: for a randomly initialised GCNN on the 4×4 lattice with $|G| = 128$,

$$\max_{g \in G,\, \boldsymbol{s}} \big|\log \psi_{\boldsymbol{\theta}}(g \cdot \boldsymbol{s}) - \log \psi_{\boldsymbol{\theta}}(\boldsymbol{s})\big| \;<\; 10^{-9}$$

in float32, i.e. at numerical precision. Symmetry is exact, not approximate.

Code: `nqs/groups.py`, `nqs.ansatz.GCNN`.

7 Putting it together: the training loop

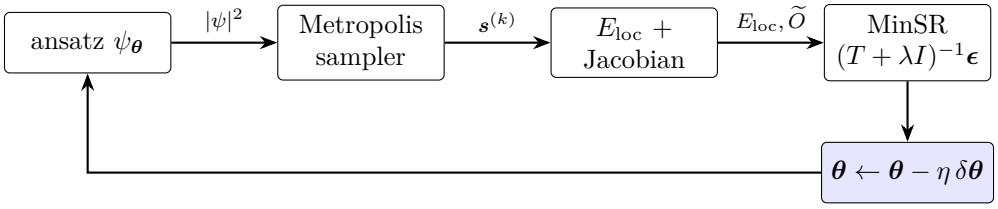
The whole VMC–MinSR loop is the following three-line update, repeated for N_{steps} iterations:

1. **Sample.** Run the Metropolis kernel to draw N_s configurations from $p_{\boldsymbol{\theta}}(\boldsymbol{s}) \propto |\psi_{\boldsymbol{\theta}}(\boldsymbol{s})|^2$.
2. **Local energy + Jacobian.** For each sample $\boldsymbol{s}^{(k)}$ compute $E_{\text{loc}}(\boldsymbol{s}^{(k)})$ and the row of $\widetilde{O}$ via `vmap(grad(...))`.

3. **MinSR.** Solve the $2N_s \times 2N_s$ system in (19) and apply the parameter update $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta\, \delta\boldsymbol{\theta}$.

We additionally schedule the diagonal shift λ: starting at 10^{-2} and decaying multiplicatively by 0.99 every step (clipped at 10^{-4}), which gives strongly stabilised early dynamics and unbiased late-stage convergence.

Code: `nqs.train.train()`.



8 Validation: comparison with exact diagonalisation

The package includes a sparse exact-diagonalisation reference (`nqs/ed.py`) usable up to $N \approx 22$ spins. For the provided examples we measure the variational energy averaged over the last 20 MinSR steps and report the relative error.

Example	Lattice	Ansatz	#params	#steps	E_{NQS}	rel. error
`tfim_1d.py`	1D, $L = 8$, $h/J = 1$	CNN	250	150	-10.2503	1.3×10^{-4}
`heisenberg_2d.py`	4×4	ViT	6.7 k	120	-11.130	8.8×10^{-3}
`heisenberg_2d_gcnn.py`	4×4	GCNN	4.7 k	80	-11.2275	9.0×10^{-5}

The GCNN reaches an accuracy roughly two orders of magnitude better than the plain ViT, with *fewer* parameters and *fewer* steps. This is the empirical payoff of building the lattice symmetries into the architecture rather than hoping the optimiser will find them.

9 Code map

File	Role
`nqs/lattices.py`	1D chain, 2D square; bonds, sublattices, translations
`nqs/groups.py`	space group $T \rtimes \{C_1, C_4, C_{4v}\}$, Cayley table
`nqs/hamiltonians.py`	TFIM and Heisenberg J_1–J_2 as sparse E_{loc} ops
`nqs/ansatz.py`	CNN, GCNN, ViT (complex log-amplitude)
`nqs/sampler.py`	Metropolis local-flip and exchange kernels
`nqs/vmc.py`	local energy + Jacobian + MinSR core
`nqs/optimizer.py`	MinSR step wrapper + Adam pre-trainer
`nqs/symmetries.py`	post-hoc translation / $\mathbb{Z}_2$ irrep projection
`nqs/train.py`	jitted training driver with diagnostics
`nqs/ed.py`	sparse exact-diagonalisation reference
`examples/`	runnable end-to-end scripts with ED comparison

References

1. G. Carleo and M. Troyer, *Solving the quantum many-body problem with artificial neural networks*, Science **355**, 602 (2017).

2. C. Roth and A. H. MacDonald, *Group convolutional neural networks improve quantum state accuracy*, Phys. Rev. B **104**, 195104 (2021).

3. C. Roth, A. Szabó and A. H. MacDonald, *High-accuracy variational Monte Carlo for frustrated magnets with deep neural networks*, Phys. Rev. B **108**, 054410 (2023).

4. L. L. Viteritti, R. Rende and F. Becca, *Transformer variational wave functions for frustrated quantum spin systems*, Phys. Rev. Lett. **130**, 236401 (2023).

5. A. Chen and M. Heyl, *Empowering deep neural quantum states through efficient optimization*, Nat. Phys. **20**, 1476 (2024).

6. S. Sorella, *Generalized Lanczos algorithm for variational quantum Monte Carlo*, Phys. Rev. B **64**, 024512 (2001).

7. T. Cohen and M. Welling, *Group equivariant convolutional networks*, ICML 2016.